

Yesterday 3:25 PM

Creational pattern
Singleton
Factory
Abstract Factory
Builder
Prototype
Structural pattern
Adapter
Decorator
Facade
Proxy
Show more ▾



Absolutely. For a **5-year Java backend interview**, don't just memorize the pattern definitions. Interviewers usually ask:

"Design this system. Which design pattern would you use and why?"

Below is a practical interview-oriented mapping and design for each system.

1. Creational Design Patterns

1. Singleton

What problem does it solve?

Ensures that **only one instance** of a class exists and provides a global access point.

Java example

 Java



```
public class ConfigurationManager {
```

```
private static volatile ConfigurationManager instance;

private ConfigurationManager() {
}

public static ConfigurationManager getInstance() {
    if (instance == null) {
        synchronized (ConfigurationManager.class) {
            if (instance == null) {
                instance = new ConfigurationManager();
            }
        }
    }
    return instance;
}
}
```

Interview points

- Private constructor
- Static instance
- Global access method
- Thread safety
- Double-checked locking
- `volatile` prevents visibility/reordering problems

Where would you use it?

Examples:

Configuration Manager
Application Metrics Registry
Cache Manager



But in Spring Boot, don't manually implement Singleton in most cases.

</> Java

```
@Service
public class PaymentService {
}
```



Spring's default bean scope is singleton.

2. Factory Pattern

Problem

You want to create objects without exposing their creation logic to the caller.

Example:

```
Notification
|
+---- EmailNotification
+---- SmsNotification
+---- WhatsAppNotification
```



Instead of:

```
</> Java

new EmailNotification()
new SmsNotification()
```



use:

```
</> Java

Notification notification =
    NotificationFactory.create("EMAIL");
```



Code

```
</> Java

interface Notification {
    void send(String message);
}
```



```
</> Java

class EmailNotification implements Notification {

    @Override
    public void send(String message) {
        System.out.println("Sending Email");
    }
}
```



```
}
}
```

</> **Java**



```
class SmsNotification implements Notification {

    @Override
    public void send(String message) {
        System.out.println("Sending SMS");
    }
}
```

</> **Java**



```
class NotificationFactory {

    public static Notification create(String type) {

        return switch (type.toUpperCase()) {
            case "EMAIL" -> new EmailNotification();
            case "SMS" -> new SmsNotification();
            default -> throw new IllegalArgumentException(
                "Unsupported notification type");
        };
    }
}
```

When to use?

When object creation depends on:

- type
- configuration
- runtime conditions

3. Abstract Factory

Factory creates **one type of object**.

Abstract Factory creates **families of related objects**.

Example:

Payment Provider



```

|
+---- Stripe
|       +-- Payment
|       +-- Refund
|
+---- Razorpay
|       +-- Payment
|       +-- Refund
  
```

</> **Java**



```

interface Payment {
    void pay();
}

interface Refund {
    void refund();
}
  
```

</> **Java**



```

interface PaymentFactory {
    Payment createPayment();
    Refund createRefund();
}
  
```

Stripe:

</> **Java**



```

class StripeFactory implements PaymentFactory {

    public Payment createPayment() {
        return new StripePayment();
    }

    public Refund createRefund() {
        return new StripeRefund();
    }
}
  
```

Razorpay:

</> Java



```
class RazorpayFactory implements PaymentFactory {

    public Payment createPayment() {
        return new RazorpayPayment();
    }

    public Refund createRefund() {
        return new RazorpayRefund();
    }
}
```

Interview distinction

Factory

Creates one product

One level

createPayment()

Abstract Factory

Creates product family

Multiple related products

createPayment() + createRefund()

4. Builder Pattern

Useful when an object has **many optional parameters**.

Instead of:

</> Java



```
new User("Manoj", "Delhi", 25, "...", "...", true);
```

use:

</> Java



```
User user = new User.Builder()
    .name("Manoj")
    .city("Delhi")
    .age(25)
```

```
.active(true)  
.build();
```

Example:

</> Java



```
class User {  
  
    private String name;  
    private String city;  
    private int age;  
    private boolean active;  
  
    private User(Builder builder) {  
        this.name = builder.name;  
        this.city = builder.city;  
        this.age = builder.age;  
        this.active = builder.active;  
    }  
  
    public static class Builder {  
  
        private String name;  
        private String city;  
        private int age;  
        private boolean active;  
  
        public Builder name(String name) {  
            this.name = name;  
            return this;  
        }  
  
        public Builder city(String city) {  
            this.city = city;  
            return this;  
        }  
  
        public Builder age(int age) {  
            this.age = age;  
            return this;  
        }  
  
        public Builder active(boolean active) {  
            this.active = active;  
            return this;  
        }  
    }  
}
```

```

        public User build() {
            return new User(this);
        }
    }
}

```

Real-world use

DTO/request construction:

</> Java



```

PaymentRequest request =
    PaymentRequest.builder()
        .amount(1000)
        .currency("INR")
        .customerId("C123")
        .build();

```

5. Prototype Pattern

Used when creating a new object from an **existing object** is easier than creating it from scratch.

Example:

Original Configuration



```

|
+---- clone for Customer A
|
+---- clone for Customer B

```

</> Java



```

class Document implements Cloneable {

    private String content;

    @Override
    public Document clone() {
        try {
            return (Document) super.clone();
        } catch (CloneNotSupportedException e) {

```

```
        throw new RuntimeException(e);  
    }  
}  
}
```

Important interview question

Prototype requires you to think about:

Shallow Copy
vs
Deep Copy



If the object contains mutable nested objects, `super.clone()` alone may not be enough.

6. Structural Design Patterns

Structural patterns answer:

How do I combine classes/objects together?

7. Adapter Pattern

Used when two incompatible interfaces need to work together.

Example:

Your application expects:

```
</> Java  
  
interface PaymentGateway {  
    void pay(double amount);  
}
```



But third-party gateway provides:

```
</> Java  
  
class RazorpayClient {  
    void makePayment(double amount) {
```



```
}
}
```

Adapter:

</> Java



```
class RazorpayAdapter implements PaymentGateway {

    private final RazorpayClient client;

    RazorpayAdapter(RazorpayClient client) {
        this.client = client;
    }

    @Override
    public void pay(double amount) {
        client.makePayment(amount);
    }
}
```

Now:

</> Java



```
PaymentGateway gateway =
    new RazorpayAdapter(new RazorpayClient());

gateway.pay(1000);
```

Think:

```
Your Interface
    ↓
  Adapter
    ↓
Third-party API
```



8. Decorator Pattern

Adds behavior without modifying the original class.

Example:

Payment
↓
LoggingDecorator
↓
RetryDecorator
↓
MetricsDecorator
↓
ActualPayment



</> **Java**



```
interface Payment {
    void pay();
}
```

</> **Java**



```
class PaymentDecorator implements Payment {

    protected Payment payment;

    PaymentDecorator(Payment payment) {
        this.payment = payment;
    }

    public void pay() {
        payment.pay();
    }
}
```

Logging:

</> **Java**



```
class LoggingDecorator extends PaymentDecorator {

    LoggingDecorator(Payment payment) {
        super(payment);
    }

    public void pay() {
        System.out.println("Payment started");

        payment.pay();
    }
}
```

```
        System.out.println("Payment completed");  
    }  
}
```

This allows behavior to be composed dynamically.

9. Facade Pattern

Provides a **simple interface over a complex subsystem**.

Suppose checkout requires:

Inventory
Payment
Shipping
Notification



Instead of controller calling all four:

</> **Java**

```
checkoutService.checkout(order);
```



Facade:

</> **Java**

```
class CheckoutFacade {  
  
    private InventoryService inventory;  
    private PaymentService payment;  
    private ShippingService shipping;  
    private NotificationService notification;  
  
    public void checkout(Order order) {  
  
        inventory.reserve(order);  
  
        payment.pay(order);  
  
        shipping.createShipment(order);  
  
        notification.send(order);  
    }  
}
```



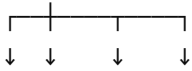
```
}
}
```

Think:

Controller



Facade



Inv Pay Ship Notification



10. Proxy Pattern

Proxy acts as a **stand-in/intermediary** for another object.

Common use cases:

Security
Caching
Logging
Lazy loading
Remote calls
Access control



Example:

</> Java

```
interface UserService {
    User getUser(long id);
}
```



</> Java

```
class UserServiceProxy implements UserService {

    private UserService target;

    public User getUser(long id) {
```



```
        if (!isAuthorized()) {  
            throw new SecurityException();  
        }  
  
        return target.getUser(id);  
    }  
  
    private boolean isAuthorized() {  
        return true;  
    }  
}
```

Spring uses proxies heavily.

For example:

</> Java



```
@Transactional  
public void transfer() {  
}
```

Spring can create a proxy around your service and apply transaction behavior.

11. Behavioral Design Patterns

Behavioral patterns focus on:

How objects communicate and how behavior is selected.

12. Strategy Pattern

One of the most important patterns for interviews.

Suppose discount calculation changes:

```
Regular Customer  
Premium Customer  
VIP Customer  
Festival Customer
```



Instead of:

</> Java



```
if (customerType.equals("PREMIUM")) {  
    ...  
} else if (...) {  
    ...  
}
```

create strategies.

</> Java



```
interface DiscountStrategy {  
    double calculateDiscount(double amount);  
}
```

</> Java



```
class RegularDiscount implements DiscountStrategy {  
  
    public double calculateDiscount(double amount) {  
        return amount * 0.05;  
    }  
}
```

</> Java



```
class PremiumDiscount implements DiscountStrategy {  
  
    public double calculateDiscount(double amount) {  
        return amount * 0.15;  
    }  
}
```

Context:

</> Java



```
class DiscountService {  
  
    private DiscountStrategy strategy;  
  
    public DiscountService(DiscountStrategy strategy) {  
        this.strategy = strategy;  
    }  
}
```

```

    public double calculate(double amount) {
        return strategy.calculateDiscount(amount);
    }
}

```

Usage:

</> **Java** 

```

DiscountService service =
    new DiscountService(new PremiumDiscount());

double discount = service.calculate(1000);

```

Main advantage

You can add:

VIPDiscount
FestivalDiscount
EmployeeDiscount 

without changing existing strategies.

13. Observer Pattern

One object changes and multiple objects need to be notified.

Example:

Order 

```

|
+---- Email Notification
+---- SMS Notification
+---- Analytics
+---- Audit

```

</> **Java** 

```

interface Observer {
    void update(Order order);
}

```

}

</> Java



```
class EmailObserver implements Observer {

    public void update(Order order) {
        System.out.println("Sending email");
    }
}
```

Subject:

</> Java



```
class Order {

    private List<Observer> observers = new ArrayList<>();

    public void addObserver(Observer observer) {
        observers.add(observer);
    }

    public void complete() {

        // complete order

        observers.forEach(o -> o.update(this));
    }
}
```

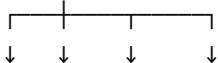
Modern distributed systems

Instead of in-memory Observer, we often use:

Order Service



Kafka / Event Bus



Email SMS Analytics Audit



That's essentially an event-driven version of the same idea.

14. Template Method

Defines the **overall algorithm**, while allowing subclasses to customize individual steps.

Example:

File Processing

```
|  
+-- Read file  
+-- Parse file  
+-- Validate  
+-- Save
```



</> Java



```
abstract class FileProcessor {  
  
    public final void process() {  
  
        readFile();  
        parse();  
        validate();  
        save();  
    }  
  
    protected abstract void readFile();  
  
    protected abstract void parse();  
  
    protected void validate() {  
        System.out.println("Common validation");  
    }  
  
    protected abstract void save();  
}
```

CSV:

</> Java



```
class CsvProcessor extends FileProcessor {  
  
    protected void readFile() {  
        System.out.println("Reading CSV");  
    }  
}
```

```
protected void parse() {
    System.out.println("Parsing CSV");
}

protected void save() {
    System.out.println("Saving CSV");
}
}
```

Key point

Parent controls the algorithm:

</> **Java**



`process()`

Child controls specific steps.

15. Design Notification System

Requirement:

Email
SMS
WhatsApp



Best patterns:

Strategy + Factory

```
NotificationService
  |
  ↓
NotificationStrategy
  |
  ├── Email
  ├── SMS
  └── WhatsApp
```



Interface:

</> **Java**



```
interface NotificationSender {  
    void send(String recipient, String message);  
}
```

Implementations:

</> Java



```
class EmailSender implements NotificationSender {  
    public void send(String recipient, String message) {  
        // email provider  
    }  
}
```

</> Java



```
class SmsSender implements NotificationSender {  
    public void send(String recipient, String message) {  
        // SMS provider  
    }  
}
```

</> Java



```
class WhatsAppSender implements NotificationSender {  
    public void send(String recipient, String message) {  
        // WhatsApp provider  
    }  
}
```

Service:

</> Java



```
class NotificationService {  
  
    private final NotificationSender sender;  
  
    NotificationService(NotificationSender sender) {  
        this.sender = sender;  
    }  
  
    public void send(String recipient, String message) {  
        sender.send(recipient, message);  
    }  
}
```

```
}  
}
```

Production design

For a real distributed system:

```
Client  
  ↓  
Notification API  
  ↓  
Notification Service  
  ↓  
Kafka  
  ↓  
Notification Workers  
  ├── Email  
  ├── SMS  
  └── WhatsApp
```



Why Kafka?

Because notification sending shouldn't necessarily block the user's request.

16. Design Payment System

Possible payment providers:

```
Stripe  
Razorpay  
PayPal
```



Use:

Strategy + Factory + Adapter

```
PaymentService  
  |  
  ↓  
PaymentStrategy  
  ├──  
  ├──  
  └──  
Stripe Razorpay PayPal
```



If third-party APIs have different interfaces:

```

PaymentStrategy
  ↓
Adapter
  ↓
Third-party SDK
  
```



Example:

</> Java



```

interface PaymentProcessor {
    PaymentResult pay(PaymentRequest request);
}
  
```

</> Java



```

class RazorpayProcessor implements PaymentProcessor {

    public PaymentResult pay(PaymentRequest request) {
        // Razorpay API
        return new PaymentResult();
    }
}
  
```

Factory:

</> Java



```

class PaymentFactory {

    public PaymentProcessor getProcessor(String provider) {

        return switch (provider) {
            case "RAZORPAY" -> new RazorpayProcessor();
            case "STRIPE" -> new StripeProcessor();
            default -> throw new IllegalArgumentException();
        };
    }
}
  
```

Production concerns

A 5-year candidate should additionally mention:

Idempotency
Transaction state
Payment timeout
Retry
Webhook handling
Duplicate payment prevention
Distributed transaction concerns
Audit
Security



Especially idempotency:

same payment request
↓
same idempotency key
↓
don't charge customer twice



17. Design File Parser

Requirements:

CSV
XML
JSON



Excellent use case for:

Strategy + Factory

FileParser
|
├──
└──
↓ ↓ ↓
CSV XML JSON



</> Java

```
interface FileParser {  
    Data parse(String content);  
}
```



</> Java



```
class CsvParser implements FileParser {
    public Data parse(String content) {
        // CSV parsing
        return new Data();
    }
}
```

</> Java



```
class JsonParser implements FileParser {
    public Data parse(String content) {
        // JSON parsing
        return new Data();
    }
}
```

Factory:

</> Java



```
class ParserFactory {

    public FileParser getParser(String type) {

        return switch (type) {
            case "CSV" -> new CsvParser();
            case "JSON" -> new JsonParser();
            case "XML" -> new XmlParser();
            default -> throw new IllegalArgumentException();
        };
    }
}
```

18. Design Logging Framework

Think:

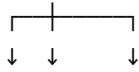
Application



Logger



LogAppender



Console File Database

Possible patterns:

Factory

Create logger/appender.

Strategy

Different output strategies.

Chain of Responsibility

For log levels:

TRACE



DEBUG



INFO



WARN



ERROR



Singleton

Potentially centralized logger configuration, though modern logging frameworks usually manage lifecycle rather than requiring you to hand-roll a Singleton.

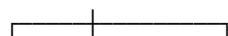
19. Design Discount Engine

This is a classic **Strategy Pattern** question.

DiscountEngine



DiscountStrategy



↓ ↓ ↓
Regular Premium Festival

But suppose multiple discounts can apply:

Base price
↓
Membership discount
↓
Coupon discount
↓
Festival discount
↓
Final price



Then **Chain of Responsibility** / **Decorator** can also be appropriate.

For example:

Discount
↓
MembershipDiscount
↓
CouponDiscount
↓
FestivalDiscount



Interview answer

"If exactly one discount algorithm is selected, I would use Strategy. If multiple discounts can be applied sequentially, I would consider a chain or composable decorators."

That's a strong 5-year-level answer.

20. Design Retry Mechanism

A retry mechanism can be implemented using:

Strategy

Different retry policies:

Fixed Delay
Exponential Backoff
Exponential Backoff + Jitter



Java



```
interface RetryStrategy {

    Duration nextDelay(int attempt);

    boolean shouldRetry(int attempt, Exception e);
}
```

Implementations:

FixedRetryStrategy
ExponentialRetryStrategy



Example:

```
Attempt 1 → fail
    ↓
100 ms
    ↓
Attempt 2 → fail
    ↓
200 ms
    ↓
Attempt 3 → fail
    ↓
400 ms
```



Important production discussion

Never blindly retry everything.

Retry:

Timeout
Temporary network failure
503
429



Usually don't retry:

Invalid request
Authentication failure
Business validation failure



Also mention:

Maximum attempts
Backoff
Jitter
Timeout
Idempotency
Circuit breaker



21. Design Rate Limiter

Common algorithms:

Fixed Window
Sliding Window
Token Bucket
Leaky Bucket



This is another excellent **Strategy Pattern** use case.

`</>` **Java**



```
interface RateLimitStrategy {  
  
    boolean allow(String clientId);  
}
```

Implement:

FixedWindowStrategy
SlidingWindowStrategy
TokenBucketStrategy



Then:

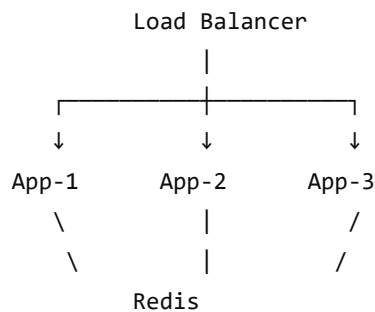
`</>` **Java**



```
class RateLimiter {  
  
    private final RateLimitStrategy strategy;  
  
    public RateLimiter(RateLimitStrategy strategy) {  
        this.strategy = strategy;  
    }  
  
    public boolean allow(String clientId) {  
        return strategy.allow(clientId);  
    }  
}
```

Distributed system

If you have multiple application instances:



You generally need shared state such as Redis, rather than keeping the counter only in each JVM.

22. Most Important Pattern Mapping for Interviews

Memorize this table:

Problem	Pattern
One instance	Singleton
Create object based on type	Factory
Create related object family	Abstract Factory
Many optional parameters	Builder
Clone existing object	Prototype

Problem	Pattern
Incompatible interfaces	Adapter
Add behavior dynamically	Decorator
Simplify complex subsystem	Facade
Intercept/control access	Proxy
Swap algorithm dynamically	Strategy
Notify many objects	Observer
Common algorithm, customizable steps	Template Method

23. The 7 System-Design Questions You Should Practice

For a 5-year Java interview, I'd practice these in this order:

1. Notification System

Strategy
Factory
Observer
Kafka
Retry



2. Payment System

Strategy
Factory
Adapter
Idempotency
Retry
Circuit Breaker



3. File Parser

Factory
Strategy
Template Method



4. Logging Framework

Factory
Strategy
Chain of Responsibility
Singleton



5. Discount Engine

Strategy
Decorator
Chain of Responsibility



6. Retry Framework

Strategy
Decorator
Exponential Backoff
Jitter
Circuit Breaker



7. Rate Limiter

Strategy
Token Bucket
Redis
Atomic operations
Distributed consistency



The key interview skill isn't saying "this is Strategy." Explain the reason:

"I would use Strategy because the core algorithm varies independently and I want to add new implementations without modifying the existing service, following the Open/Closed Principle."

That kind of explanation is what makes the answer sound like a **5-year experienced Java developer**, rather than someone who has memorized design-pattern definitions.



Yesterday 4:26 PM

Design 1

Design an Order Management System using Spring Boot.

Design 2

Design a payment system.

Design 3

Design a notification system supporting:

Show more ▾

Yes. These **7 designs** are excellent for a 5-year Java/Spring Boot interview because they combine OOP/design patterns with Spring Boot, microservices, concurrency, database design, Kafka, Redis, resilience, and scalability.

I'll show you how you should **answer each one in an interview**, not just give class diagrams.

Design 1 — Order Management System using Spring Boot

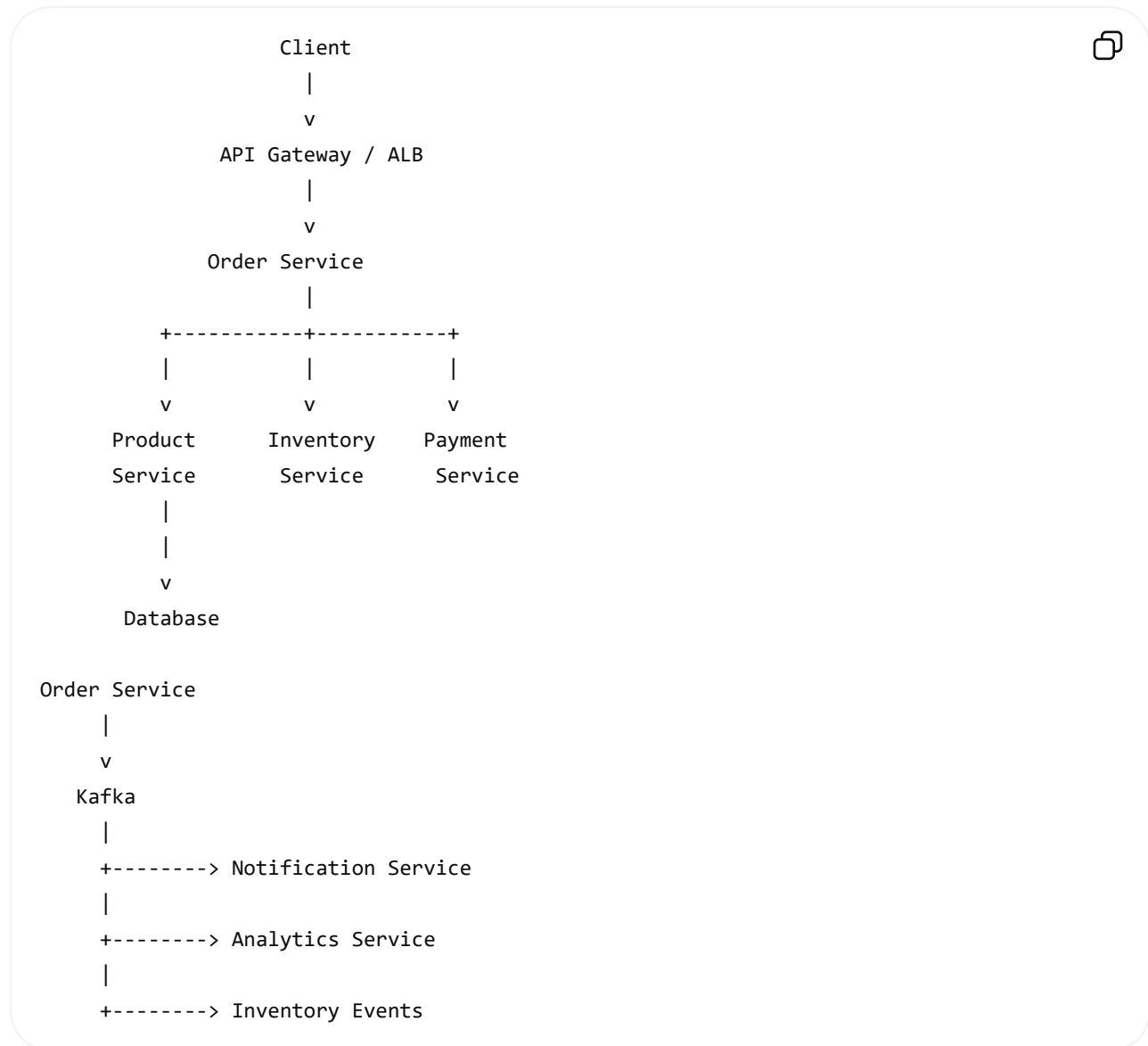
1. Requirements

Assume the system needs to:

- Create an order
- Get order details
- Update/cancel order
- Validate products
- Check inventory
- Process payment
- Track order status
- Send notification

- Handle high traffic
- Prevent duplicate orders

2. High-Level Architecture



For a simpler interview, you can initially say:

"I would start with an Order Service and separate downstream services where independent scaling or ownership is required."

3. Order API

Create Order

</> http



POST /api/orders

Request:

</> JSON



```
{
  "customerId": 101,
  "items": [
    {
      "productId": 1001,
      "quantity": 2
    }
  ]
}
```

Response:

</> JSON



```
{
  "orderId": "ORD-10001",
  "status": "PENDING"
}
```

4. Domain Model

</> Java



```
@Entity
public class Order {

    @Id
    private Long id;

    private Long customerId;

    @Enumerated(EnumType.STRING)
    private OrderStatus status;

    private BigDecimal totalAmount;
```

```
private Instant createdAt;
}
```

</> **Java**



```
public enum OrderStatus {

    CREATED,
    PAYMENT_PENDING,
    PAID,
    PROCESSING,
    SHIPPED,
    DELIVERED,
    CANCELLED

}
```

Order items:

</> **Java**



```
@Entity
public class OrderItem {

    @Id
    private Long id;

    private Long orderId;
    private Long productId;
    private int quantity;
    private BigDecimal price;

}
```

5. Spring Boot Layers

Controller

|

v

OrderService

|

+---- InventoryClient

|

+---- PaymentClient

|

+---- OrderRepository



```
|  
+---- KafkaProducer
```

Controller:

</> **Java**



```
@RestController  
@RequestMapping("/orders")  
public class OrderController {  
  
    private final OrderService orderService;  
  
    @PostMapping  
    public OrderResponse create(  
        @RequestBody CreateOrderRequest request) {  
  
        return orderService.createOrder(request);  
    }  
}
```

Service:

</> **Java**



```
@Service  
public class OrderService {  
  
    public OrderResponse createOrder(CreateOrderRequest request) {  
  
        // validate request  
  
        // check inventory  
  
        // create order  
  
        // persist order  
  
        // publish event  
  
        return response;  
    }  
}
```

6. Important Interview Question

What happens if payment succeeds but order update fails?

This is a **distributed transaction problem**.

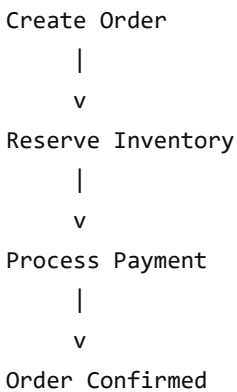
Don't say:

"I'll use @Transactional."

@Transactional works within the transaction boundary of the resource it manages; it doesn't magically make separate microservices one atomic transaction.

Instead, discuss:

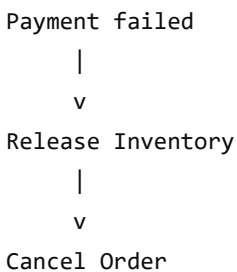
Saga Pattern



```
graph TD; A[Create Order] --> B[Reserve Inventory]; B --> C[Process Payment]; C --> D[Order Confirmed];
```



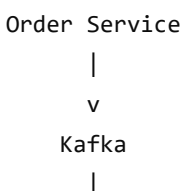
Failure:



```
graph TD; A[Payment failed] --> B[Release Inventory]; B --> C[Cancel Order];
```



For asynchronous flows:



```
graph TD; A[Order Service] --> B[Kafka];
```



```
+---- Inventory
|
+---- Payment
```

7. Prevent Duplicate Orders

Use idempotency key.

```
</> http
```



```
POST /orders
```

```
Idempotency-Key: abc123
```

Store:

```
idempotency_key
```



```
customer_id
```

```
order_id
```

```
response
```

If the same request arrives again:

```
abc123 already exists
```



```
↓
```

```
return existing response
```

This is very important in payment/order interviews.

Design 2 — Payment System

This is probably one of the most important designs.

1. Requirements

Support:

```
Razorpay
```



```
Stripe
```

```
PayPal
```

Need:

- Payment
- Refund
- Payment status
- Retry
- Timeout
- Idempotency
- Webhooks
- Audit
- Duplicate prevention

2. Architecture

Client

|

v

Payment API

|

v

Payment Service

|

+---- Payment Factory

|

+---- Razorpay Adapter

|

+---- Stripe Adapter

|

+---- PayPal Adapter

|

v

Payment DB



3. Strategy Pattern

</> Java

```
public interface PaymentProcessor {  
  
    PaymentResult pay(PaymentRequest request);  
}
```



```
RefundResult refund(RefundRequest request);
}
```

Implementations:

</> Java



```
class RazorpayProcessor implements PaymentProcessor {
}
```

</> Java



```
class StripeProcessor implements PaymentProcessor {
}
```

4. Adapter Pattern

Third-party APIs won't necessarily have the same interface.

PaymentProcessor

|

v

RazorpayAdapter

|

v

Razorpay SDK



Your application doesn't care about the third-party API structure.

5. Factory

</> Java



```
@Component
public class PaymentProcessorFactory {

    private final Map<String, PaymentProcessor> processors;

    public PaymentProcessor getProcessor(String provider) {
        PaymentProcessor processor = processors.get(provider);
    }
}
```

```

        if (processor == null) {
            throw new IllegalArgumentException(
                "Unsupported payment provider");
        }

        return processor;
    }
}

```

With Spring, this is often cleaner than a huge `switch` .

6. Payment State Machine

```

    CREATED
      |
      v
    PROCESSING
      |
      +-----> SUCCESS
      |
      +-----> FAILED

```



Refund:

```

    SUCCESS
      |
      v
    REFUND_PENDING
      |
      v
    REFUNDED

```



Don't allow invalid transitions.

For example:

```

    FAILED → REFUNDED

```



shouldn't normally be valid.

7. Idempotency

Suppose:

```
User clicks Pay
|
+--> Request 1
|
+--> Request 2
```



Without idempotency:

```
₹1000 charged
₹1000 charged
```



With idempotency:

```
Idempotency-Key = XYZ

Request 1 → process
Request 2 → return previous result
```



8. Webhooks

Payment providers may asynchronously tell you:

```
payment.success
payment.failed
refund.completed
```



Architecture:

```
Razorpay
|
| webhook
v
Payment Service
|
v
Update payment status
```



Never rely only on the synchronous payment API response.

Design 3 — Notification System

Requirements:

Email
SMS
WhatsApp



1. Architecture

```
Order Service
  |
  v
Kafka
  |
  v
Notification Service
  |
  v
Notification Factory
  |
+-----+-----+-----+
|         |         |         |
Email     SMS     WhatsApp
Provider  Provider Provider
```



2. Interface

```
</> Java

public interface NotificationSender {

    void send(NotificationRequest request);
}
```



Implementations:

```
</> Java
```



```
@Component
class EmailSender implements NotificationSender {
}
```

</> Java



```
@Component
class SmsSender implements NotificationSender {
}
```

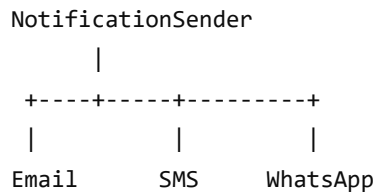
</> Java



```
@Component
class WhatsAppSender implements NotificationSender {
}
```

3. Strategy Pattern

Each channel represents a strategy.



4. Factory

</> Java



```
@Component
public class NotificationFactory {

    private final Map<NotificationType, NotificationSender> senders;

    public NotificationSender getSender(NotificationType type) {
        return senders.get(type);
    }
}
```

5. Why Kafka?

Imagine:

```
Order Service
  |
  v
sendEmail()
sendSMS()
sendWhatsApp()
```



The order API now waits for all notifications.

Instead:

```
Order Service
  |
  v
Kafka
  |
+----- Notification Worker
```



Order processing doesn't have to wait for notification delivery.

6. Failure Handling

Suppose SMS provider is down.

Use:

```
Retry
Dead Letter Topic
Exponential Backoff
Circuit Breaker
```



Example:

```
Kafka
  |
  v
SMS Consumer
  |
```



```
+-- success → done
|
+-- failure → retry
        |
        +-- retry 1
        +-- retry 2
        +-- retry 3
        |
        +-- DLQ
```

Design 4 — Rate Limiter

Suppose:

100 requests / minute / user



If user sends request 101:

HTTP 429 Too Many Requests



1. Algorithms

Know these:

Fixed Window
Sliding Window
Sliding Window Counter
Token Bucket
Leaky Bucket



For an API gateway, **Token Bucket** is a common choice.

2. Token Bucket

Imagine a bucket containing tokens.

Bucket capacity = 100
Refill = 10 tokens/sec



Request:

Request

|

v

Token available?

|

YES → consume token → allow

|

NO

|

v

HTTP 429



3. Distributed Architecture

Don't keep counters only in Java memory if there are multiple instances.

Load Balancer

|

+-----+-----+

|

|

|

v

v

v

App-1

App-2

App-3

\

|

/

\

|

/

Redis



All instances share the rate-limit state.

4. Redis Atomicity

Important interview question:

What if two application instances update the same counter simultaneously?

Use atomic Redis operations / Lua scripting where the rate-limiting algorithm requires multiple operations to be executed atomically.

5. Strategy Pattern

</> Java



```
interface RateLimitStrategy {

    boolean allow(String clientId);

}
```

Implement:

```
TokenBucketStrategy
SlidingWindowStrategy
FixedWindowStrategy
```



Now algorithm can be changed without changing RateLimiter .

Design 5 — High-Traffic Order Processing

This is a **system design** + **scalability** question.

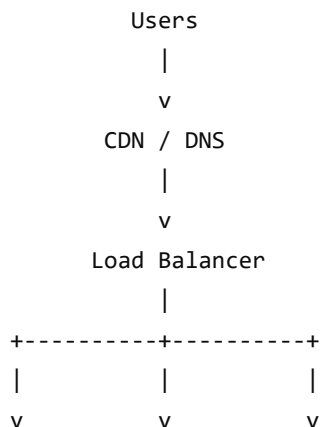
Suppose:

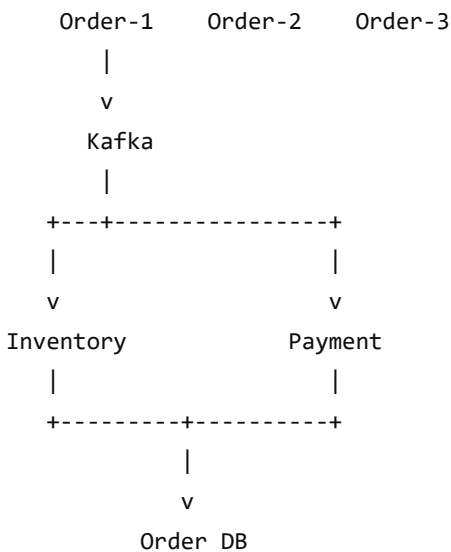
```
Normal traffic = 1,000 orders/sec
Peak traffic = 50,000 orders/sec
```



Don't synchronously perform everything inside one HTTP request.

1. Architecture





2. Why Kafka?

Suppose 50,000 requests/sec arrive.

Without a queue:

```

graph TD
    A[50k requests] --> B[DB]
    B --> C[DB overloaded]
  
```



With Kafka:

```

graph TD
    A[50k requests] --> B[Kafka]
    B --> C[Consumers process at sustainable rate]
  
```



Kafka acts as a **buffer** between producers and consumers.

3. Partitioning

Suppose:

orders topic



Partition 0

Partition 1

Partition 2

Partition 3

...

Multiple consumers process partitions concurrently.

Important:

Ordering is guaranteed within a Kafka partition, not globally across all partitions.

If ordering by `orderId` is important, use the appropriate key so related events consistently map to the same partition.

4. Database Scaling

For very high traffic:

Application

|

v

Connection Pool

|

v

Database



Potentially introduce:

Read Replica

Sharding

Partitioning

Caching

Indexes



Don't immediately say "use sharding."

First identify the bottleneck.

5. Backpressure

If consumers process:

10k/sec



but producers generate:

50k/sec



Kafka lag increases.

Monitor:

Consumer lag
Queue depth
Processing latency
Error rate
DB connection pool
CPU



Then scale consumers if downstream systems can handle it.

Design 6 — Retry Mechanism

This is a very common microservices question.

1. Basic Design

```
Service A
|
v
Service B
|
+---- success
|
+---- failure
      |
      v
    Retry
```



2. Retry Policy

Interface:

</> Java



```
public interface RetryPolicy {  
  
    boolean shouldRetry(int attempt, Exception exception);  
  
    Duration getDelay(int attempt);  
}
```

Implementations:

FixedRetryPolicy
ExponentialBackoffPolicy



3. Exponential Backoff

Example:

Attempt 1 → 100ms
Attempt 2 → 200ms
Attempt 3 → 400ms
Attempt 4 → 800ms



Formula:

$\text{delay} = \text{initialDelay} \times 2^{(\text{attempt} - 1)}$



But add **jitter** in distributed systems.

Why?

Suppose 10,000 requests fail simultaneously.

Without jitter:

All retry after 1 sec
↓



Downstream gets 10,000 requests



Fails again

With jitter:

Retry times spread out



Less traffic spike



4. What Should NOT Be Retried?

Usually don't blindly retry:

400 Bad Request

401 Unauthorized

403 Forbidden

Business validation failure



Potential retry candidates:

Timeout

503

Temporary network failure

429



The exact policy depends on the API semantics.

5. Retry + Circuit Breaker

Very important.

Don't do:

Service down



Retry

Retry

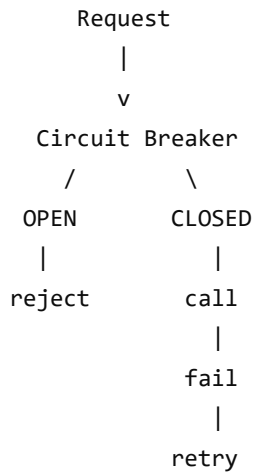
Retry



Retry
Retry

This can make the outage worse.

Instead:



After repeated failures:

CLOSED
↓
OPEN

Later:

OPEN
↓
HALF_OPEN
↓
test request
↓
success → CLOSED
failure → OPEN

Design 7 — File Processing for Millions of Records

This is where **batch processing**, **streaming**, **memory management**, **parallelism**, and **database optimization** become important.

1. Problem

Suppose:

CSV file
500 million records



Bad approach:

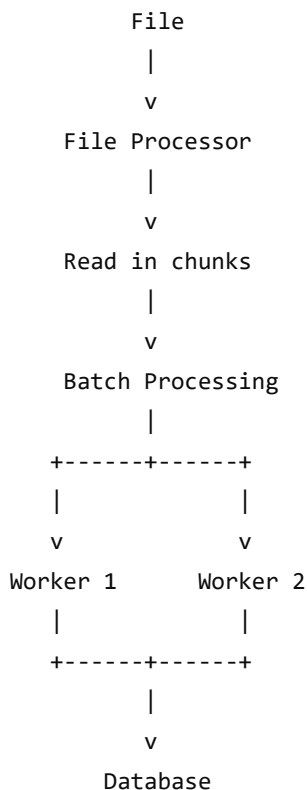
`</> Java`

```
List<Record> records = readEntireFile();
```



This can cause huge memory consumption.

2. Better Architecture



3. Spring Batch

For millions of records, Spring Batch is a strong choice.

Conceptually:

```

Job
|
+-- Step
    |
    +-- ItemReader
    |
    +-- ItemProcessor
    |
    +-- ItemWriter
  
```



Example:

```

CSV
↓
FlatFileItemReader
↓
Processor
↓
JdbcBatchItemWriter
↓
Database
  
```



4. Chunk Processing

Instead of:

```

500 million records
↓
memory
  
```



use:

```

Read 1000
↓
Process 1000
↓
Write 1000
↓
Commit
  
```





Read next 1000

For example:

Chunk size = 1000



Records 1-1000

Records 1001-2000

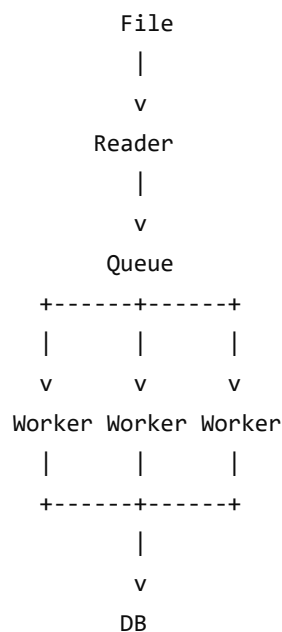
Records 2001-3000

...

The ideal chunk size depends on the workload and should be benchmarked.

5. Parallel Processing

Possible architecture:



But don't create unlimited threads.

If:

DB can handle 20 concurrent writes



don't create:

500 workers



Otherwise you'll simply move the bottleneck to the database.

6. Database Optimization

For millions of records:

Avoid

```
INSERT  
INSERT  
INSERT  
INSERT
```



Prefer batch inserts:

```
INSERT 1000 records
```



Also consider:

```
Indexes  
Batch size  
Transactions  
Connection pool  
Partitioning  
Upsert strategy  
Staging tables
```



7. Restartability

One of the biggest reasons to use Spring Batch.

Suppose:

```
Processed 700 million?
```



Imagine processing fails at record 7,000,001.

You don't want to start from zero.

You want:

Checkpoint

↓

Restart

↓

continue from previous successful point

8. Failure Handling

Use:

Retry

Skip

Dead-letter/error file

Checkpoint

Restart

Example:

Record

|

+-- valid → process

|

+-- temporary failure → retry

|

+-- invalid → skip + error file

9. The Pattern Mapping You Should Remember

These seven designs map very nicely to the patterns you listed:

Design	Primary Patterns / Concepts
Order Management	Facade, Strategy, Observer, Saga
Payment	Strategy, Factory, Adapter, State

Design	Primary Patterns / Concepts
Notification	Strategy, Factory, Observer
Rate Limiter	Strategy + Token Bucket
High-Traffic Orders	Producer/Consumer, Observer/Event-driven, Strategy
Retry	Strategy + Decorator + Circuit Breaker
File Processing	Template Method + Strategy + Batch Processing

How I Would Answer in a Real Interview

For any one of these designs, follow this sequence:

1. Clarify requirements

↓

2. Define APIs

↓

3. Identify entities

↓

4. Design high-level architecture

↓

5. Define interfaces/classes

↓

6. Apply design patterns

↓

7. Database design

↓

8. Concurrency

↓

9. Failure handling

↓

10. Scalability

↓

11. Observability

↓

12. Security



For example, if the interviewer says:

"Design a payment system."

Don't immediately start writing classes.

Start with:

"I'll first clarify the functional and non-functional requirements. I assume we need payment initiation, status tracking, refunds, multiple payment providers, idempotency, retries, webhook handling and auditability."

Then architecture.

Then:

"For multiple payment providers I'll use Strategy + Adapter. Factory/Spring DI will select the provider. For duplicate payment prevention I'll use idempotency keys. Since payment is a distributed workflow, I won't rely on a single database transaction across services."

That is the level of answer I'd target for a 5-year Java/Spring Boot interview.



ChatGPT can make mistakes. Check important info.